

Data corrections and JSON types

User

We will be correcting and adjusting the data about must watch , favourites, also remember me, and email notifications. first explain to me the difference between jsonb or json?

Assistant

JSON and JSONB are both PostgreSQL types for storing JSON data, but they differ in storage and performance:

JSON stores an exact text copy of the input, including whitespace and key order. It re-parses on every query, so it's slower to query but faster to write (no processing needed) and preserves the original formatting.

JSONB stores data in a decomposed binary format. It strips insignificant whitespace, doesn't preserve key order, and removes duplicate keys (keeping the last one). It's slower to write (has to convert on insert) but much faster to query, and it supports indexing (GIN indexes) plus richer operators (@>, ?, #>, etc.).

For this case (must-watch, favourites, remember-me, email-notification settings), JSONB is almost always the right call unless exact input formatting must be preserved.

Want the schema changes for these fields next, tutorial-style?

User

yes please. Every time they set a favourites, we need to update that in the database. sign in and out , we need to updated that item as well

Assistant

Good news first: favourites/mustWatch already write to the database on every change — that path is already wired correctly in `moviesContext.tsx`, nothing to fix there.

For remember_me and email_notifications, added a new `profiles` table (plain boolean columns, not JSONB — good contrast to the JSON/JSONB question, since these are flat known values vs. the flexible document shape in `user_movie_data`). The tutorial walks through: the SQL + RLS policies, a `UserProfile` type, an `upsertProfile` helper, and the two diffs — `remember_me` recorded in `loginUser`, `email_notifications` recorded in `registerUser`.

One thing to flag: recording `remember_me` doesn't by itself change session length — that needs a separate change to `supabase-client.ts` if the checkbox should actually control it.

User

step 4. auth-api is no longer in use. correct that

Assistant

Right — `auth-api.ts` is dead code (nothing imports it; `authContext.tsx` and `signUp.tsx` both pull from `supabase-api.ts`). Fixed Steps 4 and 5 to target the file that's actually live. Steps 4 and 5 now correctly point at `supabase-api.ts` (where `loginUser/registerUser` actually live and run), not the unused `auth-api.ts`. No other references to that file remain in the tutorial.

User

check my files, something is not right

Assistant

Found it — Steps 4 and 5 got applied to the wrong function and merged together. Two separate bugs in `loginUser` (lines 21-43):

1. Line 22 calls `supabase.auth.signUp` instead of `supabase.auth.signInWithPassword`. `loginUser` is now trying to create a new account every time someone signs in, instead of authenticating an existing one. Existing users will get a “user already registered” error (or worse) instead of logging in.
2. The `email_notifications` block from Step 5 landed in `loginUser`, not `registerUser`. `SignInProps` doesn't even have an `isSubscribed` field, so `data.isSubscribed` on line 31 is undefined in this context — and `registerUser` above never got the `upsertProfile` call it was supposed to get, so email notifications never save on sign-up.
3. The old guard `if (!authData.session || !authData.user) throw ...` got deleted. Without it, `authData.user.id` and `authData.session.access_token` will throw a runtime `TypeError` any time `authData.user` or `.session` comes back empty — now more likely since it's calling `signUp`, not `sign-in`.

```
export const registerUser = async (data: SignUpProps): Promise<void> => {
- const { error } = await supabase.auth.signUp({
+ const { data: authData, error } = await supabase.auth.signUp({
  email: data.email,
  password: data.password,
  options: {
    data: {
      full_name: data.fullName,
      is_subscribed: data.isSubscribed ?? false,
    },
  },
});
if (error) {
  throw new Error(error.message);
}
+ if (authData.user) {
+   await upsertProfile(authData.user.id, { email_notifications: data.isSubscribed ?? false });
+ }
};

export const loginUser = async (data: SignInProps): Promise<AuthSession> => {
- const { data: authData, error } = await supabase.auth.signUp({
+ const { data: authData, error } = await supabase.auth.signInWithPassword({
  email: data.email,
  password: data.password,
});
if (error) {
  throw new Error(error.message);
}
- if (authData.user) {
-   await upsertProfile(authData.user.id, { email_notifications: data.isSubscribed ?? false });
- }
-
- await upsertProfile(authData.user.id, { remember_me: data.remember_me ?? false });
+ if (!authData.session || !authData.user) {
+   throw new Error("Login failed: no session returned");
+ }
```

```
+
+ await upsertProfile(authData.user.id, { remember_me: data.remember_me ?? false });

return {
  user: {
    id: authData.user.id,
    email: authData.user.email ?? "",
  },
  access_token: authData.session.access_token,
};
};
```

Separately — worth knowing but not urgent: every file in the repo (66 of 76 tracked files) now shows as modified in `git diff`, but with `--ignore-all-space` almost all of that disappears. The working copy has CRLF line endings while everything committed in git has LF — likely the editor or OS auto-converting on save; not from the tutorial edits. Won't break the app, but will make every future diff/PR noisy and could cause fake merge conflicts. A `.gitattributes` with `* text=auto eol=lf` plus re-normalizing would fix it, separately from the login fix above.

User

permission denied for table profiles ?

Assistant

“Permission denied for table X” is a different failure than an RLS policy rejection — RLS violations say “new row violates row-level security policy,” this one means the `authenticated` role has no table-level grant on `profiles` at all. That happens when a table is created via SQL Editor and the grant Supabase normally applies to new tables doesn't land (didn't affect `user_movie_data` if that one was created differently, e.g. through the Table Editor UI).

Fix — run this in Supabase SQL Editor:

```
grant select, insert, update on public.profiles to authenticated;
```

Then verify:

```
select grantee, privilege_type
from information_schema.role_table_grants
where table_name = 'profiles';
```

Should see `authenticated` listed with `SELECT`, `INSERT`, and `UPDATE`. Also worth double-checking the RLS policies actually exist (in case the table got recreated at some point without them):

```
select policymname, cmd from pg_policies where tablename = 'profiles';
```

Should list three: `select own profile`, `insert own profile`, `update own profile`.

Once the grant is in place, try signing in/up again.

User

Ok, remember me and email notifications are ok. now Favourites are not being recorded.

Assistant

Given the same root cause just showed up on `profiles`, this is almost certainly the same issue hitting `user_movie_data` — that table was also created straight from the SQL Editor (Step 3 of the original

Supabase tutorial), with RLS policies but no explicit table grant.

Quick way to confirm: open the browser console, click a favourite icon, and look for the error `saveMovieData failed: permission denied for table user_movie_data` (the `console.error` in `moviesContext.tsx`'s `persist` function).

Fix — Supabase SQL Editor:

```
grant select, insert, update on public.user_movie_data to authenticated;
```

Then verify:

```
select grantee, privilege_type
from information_schema.role_table_grants
where table_name = 'user_movie_data';
```

Should list `authenticated` with `SELECT`, `INSERT`, `UPDATE`.

Since this has now hit two tables in a row, worth fixing at the root so every future table doesn't need this manual step:

```
alter default privileges in schema public grant select, insert, update, delete on tables to authenticated;
```

That tells Postgres “any new table created in `public` from now on automatically grants these to `authenticated`” — won't retroactively fix existing tables (that's what the two `grant` statements above are for), but stops this from recurring on the next table.

After running the grant, refresh, add a favourite, and check the `user_movie_data` table in Supabase to confirm the row updates.